

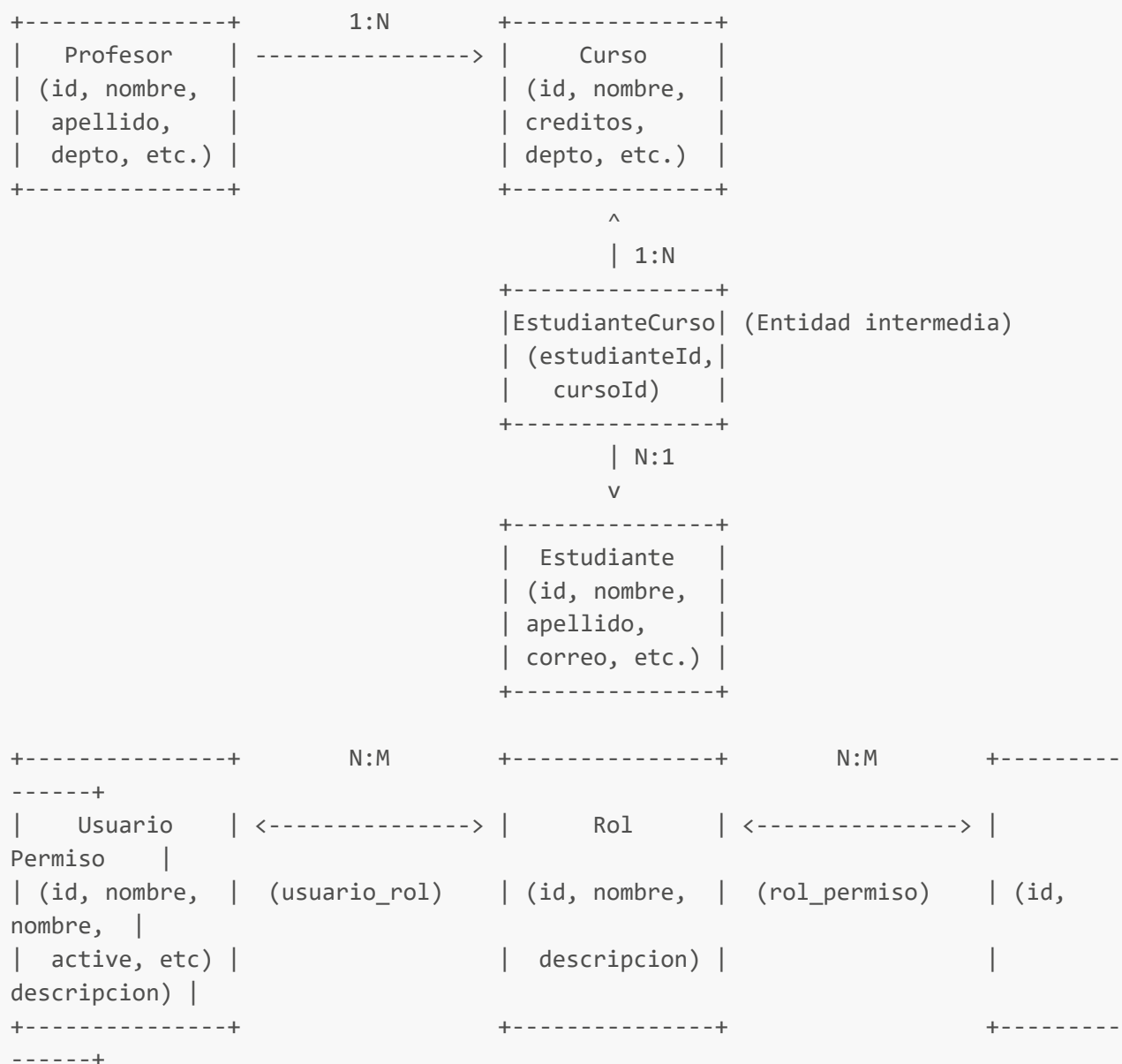
# Taller / Parcial: 5 Ejercicios de Query Methods de Complejidad Media-Alta

**Asignatura:** Computación en Internet II (Compunet II) -- Universidad ICESI

**Tema Central:** Spring Data JPA - Derived Query Methods (Navegación de Entidades, Operadores y Reglas del Parser)

**Modelo de Dominio Relacionado:** *Profesor*, *Curso*, *EstudianteCurso*, *Estudiante*, *Usuario*, *Rol* y *Permiso*

## Diagrama de Dominio y Navegación



## Matriz de los 5 Ejercicios de Parcial

| # | Repositorio Objetivo      | Temas y Operadores Evaluados                                                                                                                                                                                                                                      | Complejidad |
|---|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| 1 | ProfesorRepository        | <code>Distinct</code> , navegación 1:N ( <code>Profesor</code> -> <code>Curso</code> ), operador <code>GreaterThanOrEqualTo</code> , <code>IgnoreCase</code> , <code>ActiveTrue</code> y ordenamiento compuesto <code>OrderBy...Asc...Asc</code> .                | Media-Alta  |
| 2 | EstudianteRepository      | Navegación de 3 niveles ( <code>Estudiante</code> -> <code>EstudianteCurso</code> -> <code>Curso</code> -> <code>Profesor</code> ), sufijo <code>EndingWithIgnoreCase</code> , coincidencia anidada y ordenamiento doble.                                         | Alta        |
| 3 | EstudianteCursoRepository | Consultas sobre entidad asociativa intermedia: <code>existsBy...</code> cruzando ambos extremos ( <code>Estudiante</code> y <code>Profesor</code> ), y métricas directas con <code>countBy...</code> + <code>ContainingIgnoreCase</code> + <code>Between</code> . | Media-Alta  |
| 4 | CursoRepository           | Limitación de tamaño <code>Top&lt;N&gt;</code> / <code>First&lt;N&gt;</code> , pertenencia en colecciones <code>In</code> ( <code>Collection&lt;T&gt;</code> ), navegación inversa a intermedia y ordenamiento descendente <code>OrderBy...Desc</code> .          | Alta        |
| 5 | UsuarioRepository         | Navegación profunda en cadena N:M ( <code>Usuario</code> -> <code>Rol</code> -> <code>Permiso</code> ), operador <code>In</code> , <code>IgnoreCase</code> , filtro booleano <code>ActiveTrue</code> , modificador <code>Distinct</code> y ordenamiento múltiple. | Alta        |

## Ejercicio 1: Reporte Docente por Créditos Dictados y Departamento

### Contexto del Problema

La decanatura de la facultad necesita identificar a los profesores activos de un departamento en particular que se encuentren dictando asignaturas de alta exigencia académica (definidas como cursos con una cantidad de créditos mayor o igual a un mínimo especificado).

El reporte debe cumplir con los siguientes requerimientos:

1. **Sin duplicados:** Un profesor que dicte más de un curso que cumpla el criterio no debe aparecer repetido en la lista resultante.
2. **Ordenamiento:** Debe presentarse ordenado alfabéticamente de forma ascendente por el `apellido` del profesor y, en caso de empate, por su `nombre`.

### Requerimientos Técnicos del Query Method

- **Repositorio:** `ProfesorRepository`
- **Entidad de Consulta:** `Profesor`
- **Modificador:** `Distinct` (evita duplicados al hacer `JOIN` con la colección `cursos`).
- **Filtros requeridos:**
  - Estado del profesor: `active = true` (`ActiveTrue`).

- Departamento del profesor: coincidencia exacta insensible a mayúsculas (`DepartamentoIgnoreCase`).
  - Créditos del curso: mayor o igual a `minCreditos` (`Cursos_CreditosGreaterThanOrEqualTo`), navegando por la relación `cursos`.
  - **Ordenamiento:** `OrderByApellidoAscNombreAsc`.
- 

## Solución

### 1. Firma en el Repositorio (`ProfesorRepository.java`)

```
package com.compunet.springboot.repository;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.compunet.springboot.model.Profesor;

@Repository
public interface ProfesorRepository extends JpaRepository<Profesor, Long> {

    List<Profesor>
    findDistinctByActiveTrueAndDepartamentoIgnoreCaseAndCursos_CreditosGreaterThanOrEqualTo
    OrderByApellidoAscNombreAsc(
        String departamento,
        int minCreditos
    );
}
```

### 2. Invocación en la Capa de Servicio (`ProfesorService.java`)

```
package com.compunet.springboot.service;

import java.util.List;
import org.springframework.stereotype.Service;
import com.compunet.springboot.model.Profesor;
import com.compunet.springboot.repository.ProfesorRepository;
import lombok.RequiredArgsConstructor;

@Service
@RequiredArgsConstructor
public class ProfesorService {

    private final ProfesorRepository profesorRepository;

    public List<Profesor> listarProfesoresDestacadosPorDepartamento(String depto,
int minCreditos) {
        return profesorRepository
    }
```

```

    .findDistinctByActiveTrueAndDepartamentoIgnoreCaseAndCursos_CreditosGreaterThanOrEqualToOrderByApellidoAscNombreAsc(
        depto,
        minCreditos
    );
}
}

```

## 🔍 Análisis de la Anatomía del Método

|          |             |        |                     |      |                        |                     |                                     |                             |
|----------|-------------|--------|---------------------|------|------------------------|---------------------|-------------------------------------|-----------------------------|
| find     | Distinct    | By     | ActiveTrue          | And  | DepartamentoIgnoreCase | And                 | Cursos_CreditosGreaterThanOrEqualTo | OrderByApellidoAscNombreAsc |
| [—]      | [—]         | [—]    | [—]                 | [—]  | [—]                    | [—]                 | [—]                                 | [—]                         |
|          |             |        |                     |      |                        |                     |                                     |                             |
| Acción   | Modificador | Raíz   | Filtro 1            | Nexo |                        | Filtro 2            | Nexo                                |                             |
| Filtro 3 |             |        | Ordenamiento        |      |                        |                     |                                     |                             |
|          |             |        | (boolean)           |      |                        | (String ignoreCase) |                                     | (1:N                        |
| navegada | >=          | valor) | (Compuesto Asc/Asc) |      |                        |                     |                                     |                             |

**[!IMPORTANT] Trampa Frecuente de Parcial:** Al navegar colecciones **OneToMany** (**Profesor.cursos**), el motor relacional genera internamente un **LEFT OUTER JOIN** o **INNER JOIN**. Si un profesor tiene 3 cursos con  $\geq 4$  créditos, la consulta SQL retornará 3 filas para el mismo profesor. La palabra clave **Distinct** instruye a Hibernate/Spring Data a filtrar las entidades repetidas en el resultado.

## Ejercicio 2: Búsqueda de Estudiantes por Especialidad Docente y Dominio Institucional

### Contexto del Problema

La dirección del programa académico necesita auditar qué estudiantes activos con cuentas de correo institucionales de un dominio específico (por ejemplo, correos que terminen en "**@icesi.edu.co**") están cursando materias dictadas por docentes de una especialidad técnica dada (por ejemplo, "**Inteligencia Artificial**" o "**Ingeniería de Software**").

El resultado debe listarse sin duplicados y ordenado alfabéticamente por el **apellido** y luego por el **nombre** del estudiante.

### Requerimientos Técnicos del Query Method

- **Repositorio:** **EstudianteRepository**
- **Entidad de Consulta:** **Estudiante**
- **Ruta de Navegación Multinivel:** **Estudiante** → **estudianteCursos** → **curso** → **profesor** → **especialidad**.
- **Filtros requeridos:**

- Estado del estudiante: `active = true` (`ActiveTrue`).
  - Dominio del correo: finaliza con una cadena dada, sin distinguir mayúsculas/minúsculas (`CorreoInstitucionalEndingWithIgnoreCase`).
  - Especialidad del docente: coincidencia exacta insensible a mayúsculas (`EstudianteCursos_Curso_Profesor_EspecialidadIgnoreCase`).
  - **Modificador:** `Distinct` (evita duplicar al estudiante si cursa más de una materia con profesores de esa especialidad).
  - **Ordenamiento:** `OrderByApellidoAscNombreAsc`.
- 

## Solución

### 1. Firma en el Repositorio (`EstudianteRepository.java`)

```
package com.compunet.springboot.repository;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.compunet.springboot.model.Estudiante;

@Repository
public interface EstudianteRepository extends JpaRepository<Estudiante, Long> {

    List<Estudiante>
    findDistinctByActiveTrueAndCorreoInstitucionalEndingWithIgnoreCaseAndEstudianteCursos_Curso_Profesor_EspecialidadIgnoreCaseOrderByApellidoAscNombreAsc(
        String dominioCorreo,
        String especialidad
    );
}
```

### 2. Invocación en la Capa de Servicio (`EstudianteService.java`)

```
package com.compunet.springboot.service;

import java.util.List;
import org.springframework.stereotype.Service;
import com.compunet.springboot.model.Estudiante;
import com.compunet.springboot.repository.EstudianteRepository;
import lombok.RequiredArgsConstructor;

@Service
@RequiredArgsConstructor
public class EstudianteService {

    private final EstudianteRepository estudianteRepository;

    public List<Estudiante> buscarEstudiantesPorEspecialidadDocenteYDominio(String
```

```

dominio, String especialidad) {
    return estudianteRepository

    .findDistinctByActiveTrueAndCorreoInstitucionalEndingWithIgnoreCaseAndEstudianteCu
rsos_Curso_Profesor_EspecialidadIgnoreCaseOrderByApellidoAscNombreAsc(
        dominio,
        especialidad
    );
}
}

```

## Regla del Parser de Spring Data (Guion Bajo \_)

Cuando se encadenan propiedades de múltiples entidades

(**EstudianteCursos\_Curso\_Profesor\_Especialidad**), el uso del guion bajo **\_** actúa como delimitador léxico explícito para guiar al parser de Spring Data en la navegación por el árbol de relaciones:

Estudiante  $\xrightarrow{\text{estudianteCursos}}$  EstudianteCurso  $\xrightarrow{\text{curso}}$  Curso  $\xrightarrow{\text{profesor}}$  Profesor  $\xrightarrow{\text{especialidad}}$  String

## Ejercicio 3: Validación y Métricas sobre la Entidad Intermedia (**EstudianteCurso**)

### Contexto del Problema

Durante el proceso de matrícula académica, se deben ejecutar dos operaciones clave de alto rendimiento directamente sobre la tabla asociativa intermedia:

1. **Validación booleana instantánea:** Comprobar si existe al menos un estudiante **activo** inscrito con un profesor en particular dentro de un departamento determinado.
2. **Métrica cuantitativa:** Contar cuántas inscripciones existen en cursos cuyo nombre contenga un texto dado (ej. "Programación") y cuyos créditos se encuentren dentro de un rango inclusivo [**min**, **max**].

### Requerimientos Técnicos del Query Method

- **Repositorio:** **EstudianteCursoRepository**
- **Entidad de Consulta:** **EstudianteCurso**
- **Parte A (Existencia booleana):**
  - Palabra clave: **existsBy...** (retorna **boolean**).
  - Navegación bilateral: hacia **Estudiante** (**Estudiante\_ActiveTrue**), hacia **Curso** (**Curso\_DepartamentoIgnoreCase**) y hacia **Profesor** del curso (**Curso\_Profesor\_Id**).
- **Parte B (Conteo derivado):**
  - Palabra clave: **countBy...** (retorna **long**).
  - Filtros: **Curso\_NombreContainingIgnoreCase** y **Curso\_CreditosBetween**.

## Solución

### 1. Firmas en el Repositorio (**EstudianteCursoRepository.java**)

```
package com.compunet.springboot.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.compunet.springboot.model.EstudianteCurso;
import com.compunet.springboot.model.EstudianteCursoId;

@Repository
public interface EstudianteCursoRepository extends JpaRepository<EstudianteCurso,
EstudianteCursoId> {

    // Parte A: Verificación booleana cruzada
    boolean
existsByEstudiante_ActiveTrueAndCurso_DepartamentoIgnoreCaseAndCurso_Profesor_Id(
        String departamentoCurso,
        Long profesorId
    );

    // Parte B: Conteo derivado por subcadena y rango
    long countByCurso_NombreContainingIgnoreCaseAndCurso_CreditosBetween(
        String subcadenaNombre,
        int minCreditos,
        int maxCreditos
    );
}
```

## 2. Invocación en la Capa de Servicio (**MatriculaService.java**)

```
package com.compunet.springboot.service;

import org.springframework.stereotype.Service;
import com.compunet.springboot.repository.EstudianteCursoRepository;
import lombok.RequiredArgsConstructor;

@Service
@RequiredArgsConstructor
public class MatriculaService {

    private final EstudianteCursoRepository estudianteCursoRepository;

    public boolean tieneAlumnosActivosConProfesor(String departamento, Long
profesorId) {
        return estudianteCursoRepository

.existsByEstudiante_ActiveTrueAndCurso_DepartamentoIgnoreCaseAndCurso_Profesor_Id(
            departamento,
            profesorId
        );
    }
}
```

```
public long contarInscripcionesPorNombreYRangoCreditos(String texto, int min,
int max) {
    return estudianteCursoRepository
        .countByCurso_NombreContainingIgnoreCaseAndCurso_CreditosBetween(
            texto,
            min,
            max
        );
}
```

## Rendimiento SQL de `existsBy` y `countBy`

- **`existsBy...`**: Spring Data traduce este método a un `SELECT 1 FROM estudiante_curso ... LIMIT 1`. Retorna inmediatamente apenas encuentra la primera coincidencia sin cargar entidades completas en memoria.
- **`countBy...`**: Genera un `SELECT COUNT(*) FROM estudiante_curso ...`, ejecutando la agregación directamente en la base de datos sin transferir registros a la JVM.

## Ejercicio 4: Limitación `Top 5`, Colecciones `In` y Navegación Inversa

### Contexto del Problema

Para un informe gerencial, la universidad requiere consultar los **5 cursos con mayor cantidad de créditos** que:

1. Pertenezcan a un conjunto seleccionado de departamentos académicos (por ejemplo: `["Ingeniería", "Ciencias Básicas", "Diseño"]`).
2. Sean dictados por un profesor con un apellido en específico (insensible a mayúsculas).
3. Tengan inscrito a por lo menos un estudiante de una lista de IDs de estudiantes destacados.

El listado debe estar ordenado de mayor a menor según los créditos del curso.

### Requerimientos Técnicos del Query Method

- **Repositorio:** `CursoRepository`
- **Entidad de Consulta:** `Curso`
- **Limitación de resultados:** `findTop5By...` o `findFirst5By...`
- **Filtros requeridos:**
  - Departamento del curso en colección: `DepartamentoIn(Collection<String> departamentos)`.
  - Apellido del profesor: `Profesor_ApellidoIgnoreCase(String apellido)`.
  - ID de estudiantes matriculados: `EstudianteCursos_Estudiante_IdIn(Collection<Long> estudiantesIds)`.
- **Ordenamiento:** `OrderByCreditosDesc`.

### Solución



## 1. Firma en el Repositorio (**CursoRepository.java**)

```
package com.compunet.springboot.repository;

import java.util.Collection;
import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.compunet.springboot.modelCurso;

@Repository
public interface CursoRepository extends JpaRepository<Curso, Long> {

    List<Curso>
    findTop5ByDepartamentoInAndProfesor_ApellidoIgnoreCaseAndEstudianteCursos_Estudian
    te_IdInOrderByCreditosDesc(
        Collection<String> departamentos,
        String apellidoProfesor,
        Collection<Long> estudiantesIds
    );
}
```

## 2. Invocación en la Capa de Servicio (**CursoService.java**)

```
package com.compunet.springboot.service;

import java.util.Collection;
import java.util.List;
import org.springframework.stereotype.Service;
import com.compunet.springboot.modelCurso;
import com.compunet.springboot.repository.CursoRepository;
import lombok.RequiredArgsConstructor;

@Service
@RequiredArgsConstructor
public class CursoService {

    private final CursoRepository cursoRepository;

    public List<Curso> obtenerTop5CursosPorDepartamentosProfesorYEstudiantes(
        Collection<String> departamentos,
        String apellidoProfesor,
        Collection<Long> estudiantesIds) {

        return cursoRepository

        .findTop5ByDepartamentoInAndProfesor_ApellidoIgnoreCaseAndEstudianteCursos_Estudia
        nte_IdInOrderByCreditosDesc(
            departamentos,
            apellidoProfesor,
```

```
        estudiantesIds
    );
}
}
```

## Conceptos Clave Evaluados

- **Top<N> / First<N>**: Se traduce automáticamente a la cláusula **LIMIT <N>** en el dialecto de base de datos correspondiente (PostgreSQL, MySQL, H2).
- **Operador In con Collection<T>**: Permite evaluar listas dinámicas de parámetros y se traduce a cláusulas SQL de la forma **campo IN (?, ?, ...)**.

## Ejercicio 5: Navegación Profunda **ManyToMany** en Cadena (**Usuario** → **Rol** → **Permiso**)

### Contexto del Problema

El equipo de seguridad de la información necesita consultar los **usuarios activos** del sistema que posean al menos uno de los roles pertenecientes a una lista dada de nombres de roles (ej. `["ADMINISTRADOR", "DOCENTE_LIDER"]`), y que además dichos roles tengan asociado un permiso específico en el sistema (ej. `"MODIFICAR_NOTAS"`), sin distinguir mayúsculas ni minúsculas.

La consulta debe retornar el listado sin usuarios repetidos, ordenado alfabéticamente de forma ascendente por el **apellido** y luego por el **nombre** del usuario.

### Requerimientos Técnicos del Query Method

- **Repositorio:** `UsuarioRepository`
- **Entidad de Consulta:** `Usuario`
- **Navegación de 2 Relaciones ManyToMany:** `Usuario` → `roles` → `permisos`.
- **Filtros requeridos:**
  - Estado del usuario: `active = true` (`ActiveTrue`).
  - Nombre del rol contenido en una lista: `Roles_NombreIn(Collection<String> nombresRoles)`.
  - Nombre del permiso asignado al rol: `Roles_Permisos_NombreIgnoreCase(String nombrePermiso)`.
- **Modificador:** `Distinct` (evita duplicar usuarios que tengan múltiples roles coincidentes).
- **Ordenamiento:** `OrderByApellidoAscNombreAsc`.

### Solución

#### 1. Firma en el Repositorio (`UsuarioRepository.java`)

```
package com.compunet.springboot.repository;

import java.util.Collection;
import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.compunet.springboot.model.Usuario;

@Repository
public interface UsuarioRepository extends JpaRepository<Usuario, Long> {

    List<Usuario>
    findDistinctByActiveTrueAndRoles_NombreInAndRoles_Permisos_NombreIgnoreCaseOrderBy
    ApellidoAscNombreAsc(
        Collection<String> nombresRoles,
        String nombrePermiso
    );
}
```

## 2. Invocación en la Capa de Servicio (**UsuarioService.java**)

```
package com.compunet.springboot.service;

import java.util.Collection;
import java.util.List;
import org.springframework.stereotype.Service;
import com.compunet.springboot.model.Usuario;
import com.compunet.springboot.repository.UsuarioRepository;
import lombok.RequiredArgsConstructor;

@Service
@RequiredArgsConstructor
public class UsuarioService {

    private final UsuarioRepository usuarioRepository;

    public List<Usuario> listarUsuariosActivosPorRolesYPermiso(Collection<String>
roles, String permiso) {
        return usuarioRepository

        .findDistinctByActiveTrueAndRoles_NombreInAndRoles_Permisos_NombreIgnoreCaseOrderB
yApellidoAscNombreAsc(
            roles,
            permiso
        );
    }
}
```

## Anatomía de la Navegación **ManyToMany** Anidada

Usuario  $\xrightarrow{\text{roles (N:M)}}$  Rol  $\xrightarrow{\text{permisos (N:M)}}$  Permiso

- **Roles\_NombreIn**: Filtra en la entidad **Rol** por el campo **nombre** recibiendo una colección (**IN** (?, ?)).
- **Roles\_Permisos\_NombreIgnoreCase**: Continúa la navegación hacia la colección **permisos** dentro de **Rol** y filtra por su propiedad **nombre** ignorando mayúsculas/minúsculas.
- **Distinct**: Asegura que un usuario con 2 roles válidos sea retornado exactamente una vez.

## Tabla Maestra de Palabras Clave para el Parcial

| Palabra Clave                               | Tipo                    | Propósito                                                                                   | Ejemplo en Spring Data                         |
|---------------------------------------------|-------------------------|---------------------------------------------------------------------------------------------|------------------------------------------------|
| <b>find...By</b> / <b>get...By</b>          | Introductor             | Prefijo estándar para consultas que retornan colecciones o entidades.                       | <b>findBy...</b> , <b>getDistinctBy...</b>     |
| <b>existsBy...</b>                          | Introductor             | Consulta de existencia booleana optimizada ( <b>LIMIT 1</b> ).                              | <b>existsByEstudiante_ActiveTrueAnd...</b>     |
| <b>countBy...</b>                           | Introductor             | Conteo directo de filas ejecutado en el motor relacional.                                   | <b>countByCurso_CreditosBetween(...)</b>       |
| <b>Distinct</b>                             | Modificador             | Elimina registros repetidos producto de <b>JOIN</b> en relaciones <b>1:N</b> o <b>N:M</b> . | <b>findDistinctBy...</b>                       |
| <b>Top&lt;N&gt;</b> / <b>First&lt;N&gt;</b> | Limitador               | Limita la cantidad de registros devueltos a nivel de base de datos.                         | <b>findTop5By...</b> , <b>findFirst10By...</b> |
| <b>IgnoreCase</b>                           | Modificador de Atributo | Comparación insensible a                                                                    | <b>findByDepartamentoIgnoreCase(...)</b>       |

| Palabra Clave                    | Tipo                | Propósito                                                                         | Ejemplo en Spring Data                                  |
|----------------------------------|---------------------|-----------------------------------------------------------------------------------|---------------------------------------------------------|
|                                  |                     | mayúsculas y minúsculas ( <b>UPPER/LOWER</b> ).                                   |                                                         |
| <b>ActiveTrue / ActiveFalse</b>  | Booleano Directo    | Evalúa campos booleanos sin requerir parámetro en el método.                      | <b>findByActiveTrue(...)</b>                            |
| <b>In</b>                        | Operador Relacional | Verifica si el atributo pertenece a una <b>Collection&lt;?&gt;</b> dada.          | <b>findByDepartamentoIn(Collection&lt;String&gt; d)</b> |
| <b>Between</b>                   | Operador Relacional | Rango numérico o de fechas inclusivo [ <b>min, max</b> ].                         | <b>findByCreditosBetween(int min, int max)</b>          |
| <b>GreaterThanOrEqual</b>        | Operador Relacional | Comparación de orden mayor o igual ( <b>≥</b> ).                                  | <b>findByCreditosGreaterThanOrEqual(int min)</b>        |
| <b>EndingWith / StartingWith</b> | Coincidencia String | Coincidencia de prefijos ( <b>LIKE 'val%'</b> ) o sufijos ( <b>LIKE '%val'</b> ). | <b>findByCorreoEndingWithIgnoreCase(...)</b>            |
| <b>Containing</b>                | Coincidencia String | Coincidencia de subcadena ( <b>LIKE '%val%'</b> ).                                | <b>findByNombreContainingIgnoreCase(...)</b>            |
| <b>OrderBy...Asc / ...Desc</b>   | Cláusula de Orden   | Define el ordenamiento de los resultados (admite ordenamiento múltiple).          | <b>OrderByApellidoAscNombreAsc</b>                      |
| <b>_</b> (Guion bajo)            | Desambiguación      | Delimita niveles de                                                               | <b>EstudianteCursos_Curso_Profesor_Id</b>               |

| Palabra Clave | Tipo | Propósito                                     | Ejemplo en Spring Data |
|---------------|------|-----------------------------------------------|------------------------|
|               |      | navegación<br>entre<br>entidades<br>anidadas. |                        |